一、forms组件

二、教学目标

1、理解并实现分页功能的优化

2、掌握Forms组件的基本使用

3、运用Forms组件完成表单的渲染、数据校验和错误信息展示

4、掌握Forms组件的钩子函数

5、了解Forms组件的常用字段类型及其参数

三、完善分页

3.0.1 问题

- 现在我们看到所有的分页都展示了出来这样显然不符合我们的要求
- 目标是要显示10个就好 然后点击某个分页的时候激活标签

`views.py`

```
def fenye(request):

    book_queryset = models.Book.objects.all()

    # 获取当前页的数据
    page_num = int(request.GET.get('page',1))#默认显示第一页
    # 一页展示多少条数据
    per_page = 50
    # 每页的起始位置和结束位置
    #起始位置 = (当前页 - 1) × 每页数量
    #结束位置 = 当前页 × 每页数量
    start_num = (page_num-1) * per_page
    end_num = page_num * per_page
    #总页数
    total_num = book_queryset.count() #总条数
    # 总页数 = (总数据量 + 每页数量 - 1) // 每页数量（向上取整算法）
    #200条记录/每页50条 → (200+50-1)//50 = 249//50 = 4页
    total_page = (total_num + per_page -1) // per_page #总页数

    #存储当前页
    #temp = page_num

    # #出现负数怎么办?
    # if page_num < 6:
```

```
    #     page_num =6
    # #当我的 当前页 大于总页数-4的时候 说明到了最后的标签了
    # elif page_num > total_page-4:
    #     page_num = total_page-4


    #始终显示10个页码（或更少）
    #当前页保持在中间位置（前5后4）
    #处理首尾边界情况
    page_html = ''
    #总页数不超过10页时
    if total_page <= 10:
        # 总页数<=10时显示所有页码
        for i in range(1, total_page + 1):
            # 当前页添加active类高亮显示
            active = 'active' if page_num == i else ''
            page_html += f'<li class="page-item {active}"><a class="page-link"
href="?page={i}">{i}</a></li>'
    #总页数超过10页时
    else:
        # 正确的动态页码范围算法
        # 确保当前页在中间，且范围不超过总页数
        # 计算页码导航显示范围（核心算法）：
        # start_range: 保证显示10个页码的前提下，动态计算起始页码
        #  min(page_num - 5, total_page - 9): 当前页尽量居中（前5后4）
        #  max(1, ...): 保证起始页码不小于1
        start_range = max(1, min(page_num - 5, total_page - 9))
        # end_range: 根据start_range计算结束页码（最多显示10个页码）
        # min(start_range + 9, total_page): 保证结束页码不超过总页数
        end_range = min(start_range + 9, total_page)

        for i in range(start_range, end_range + 1):
            active = 'active' if page_num == i else ''
            page_html += f'<li class="page-item {active}"><a class="page-link"
href="?page={i}">{i}</a></li>'

    # 获取当前页数据（按每页显示条数切片）
    book_queryset = book_queryset[start_num:end_num]
    return render(request, 'fenye.html', locals())
```

3.0.1 扩展（自行学习）

当然我们djaong当中还有分页组件Django Paginator

Paginator 对象属性

- `count`：总记录数
- `num_pages`：总页数
- `page_range`：页码范围（从1开始的迭代器）

Page 对象属性

- `object_list`：当前页的对象列表
- `number`：当前页码
- `has_previous()`：是否有上一页
- `has_next()`：是否有下一页

- `previous_page_number()`：上一页页码
- `next_page_number()`：下一页页码
- `start_index()`：当前页第一条记录的序号
- `end_index()`：当前页最后一条记录的序号

urls.py

```
path('fenye2/',views.fenye2)
```

app01 - > views.py

```
from django.core.paginator import Paginator, EmptyPage, PageNotAnInteger


def fenye2(request):
    # 获取所有书籍数据
    book_queryset = models.Book.objects.all()

    # 每页显示50条记录
    per_page = 50

    # 创建Paginator对象（参数：数据集，每页数量）
    paginator = Paginator(book_queryset, per_page)

    # 从请求参数获取当前页码（默认为1）
    page_num = request.GET.get('page', 1)

    try:
        # 获取当前页对象
        page_obj = paginator.page(page_num)
    except PageNotAnInteger:
        # 如果page参数不是整数，返回第一页
        page_obj = paginator.page(1)
    except EmptyPage:
        # 如果页码超出范围，返回最后一页
        page_obj = paginator.page(paginator.num_pages)

    # 计算页码范围
    # 获取当前页前后各5页的范围
    index = page_obj.number  # 当前页码
    max_index = paginator.num_pages  # 总页数

    # 计算页码范围（最多显示11个页码）
    if max_index <= 11:
        page_range = paginator.page_range
    else:
        # 确保页码范围不超出1到max_index
        start_index = max(1, index - 5)
        end_index = min(max_index, index + 5)

        # 如果起始页码太靠后，调整显示范围
        if end_index - start_index < 10:
            if index < 6:
                end_index = min(11, max_index)
            else:
                start_index = max(1, max_index - 10)
```

```
        page_range = range(start_index, end_index + 1)

    # 返回结果
    return render(request, 'fenye2.html', {
        'page_obj': page_obj,  # 当前页对象
        'paginator': paginator,  # 分页器对象
        'page_range': page_range,  # 页码范围
    })
```

fenye2.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>


    {% load static %}

    <!-- 导入外部的css-->
    <link rel="stylesheet" href={% static 'bootstrap-3.4.1-
dist/css/bootstrap.min.css' %}>
    <!-- 导入外部的js-->
    <script src={% static "js/jquery-3.7.0.min.js" %}></script>
    <!-- 导入外部的js-->
    <script src={% static "bootstrap-3.4.1-dist/js/bootstrap.min.js" %}>
</script>

</head>
<body>
    <div class="container">
        <h1 class="mt-4 mb-4">图书列表</h1>

        <!-- 显示当前页数据 -->
        {% for book_obj in page_obj %}
            <div class="card mb-2">
                <div class="card-body">
                    {{ book_obj.title }}
                </div>
            </div>
        {% endfor %}

        <!-- 分页导航 -->
        <nav aria-label="Page navigation">
            <ul class="pagination justify-content-center">
                <!-- 上一页按钮 -->
                <li class="page-item {% if not page_obj.has_previous
%}disabled{% endif %}">
                    <a class="page-link"
                       href="?page={% if page_obj.has_previous %}{{
page_obj.previous_page_number }}{% else %}1{% endif %}"
                       aria-label="Previous">
                        <span aria-hidden="true">&laquo;</span>
                    </a>
                </li>
```

```
                <!-- 首页按钮 -->
                {% if page_obj.number > 6 %}
                    <li class="page-item">
                        <a class="page-link" href="?page=1">1</a>
                    </li>
                    <li class="page-item disabled">
                        <span class="page-link">...</span>
                    </li>
                {% endif %}

                <!-- 中间页码 -->
                {% for num in page_range %}
                    <li class="page-item {% if num == page_obj.number %}active{%
endif %}">
                        <a class="page-link" href="?page={{ num }}">{{ num }}
</a>
                    </li>
                {% endfor %}

                <!-- 末页按钮 -->
                {% if page_obj.number < paginator.num_pages|add:-5 %}
                    <li class="page-item disabled">
                        <span class="page-link">...</span>
                    </li>
                    <li class="page-item">
                        <a class="page-link" href="?page={{ paginator.num_pages
}}">{{ paginator.num_pages }}</a>
                    </li>
                {% endif %}

                <!-- 下一页按钮 -->
                <li class="page-item {% if not page_obj.has_next %}disabled{%
endif %}">
                    <a class="page-link"
                       href="?page={% if page_obj.has_next %}{{
page_obj.next_page_number }}{% else %}{{ paginator.num_pages }}{% endif %}"
                       aria-label="Next">
                        <span aria-hidden="true">&raquo;</span>
                    </a>
                </li>
            </ul>
        </nav>


    </div>
</body>
</html>
```

四、forms组件

4.0.1 注册功能

获取用户名和密码,利用 form 表单提交数据,在后端进行判断,用户名和密码要符合以下条件:

- 1.用户名不能包含 666
- 2.密码不能少于3位

最后将提示信息展示到前端页面

新建项目

django_forms

自行创建

urls.py

```
path('test/',views.test)
```

views.py

```
def test(request):
    back_dict = {'username':'','password':''}
    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')
        if '666' in username:
            #return HttpResponse('用户名不能包含666')
            back_dict['username'] = '用户名不能包含666'
        if len(password) < 3:
            #return HttpResponse('密码不能小于3位')
            back_dict['password'] = '密码不能小于3位'

    return render(request,'test.html',locals())
```

test.html

```
<body>

<form action="" method="post">
    <p>用户名: <input type="text" name="username">
        <span style="color: red">{{ back_dict.username }}</span>
    </p>
    <p>密码: <input type="text" name="password">
        <span style="color: red">{{ back_dict.password }}</span>
    </p>
    <input type="submit">
</form>
</body>
```

总结一下实现上述功能用到了哪些知识点:

1.手动书写前端 form 表单代码(渲染 html 页面)

2.后端对用户数据进行校验(数据校验)

3.对不符合要求的数据展示在前端提示信息(展示提示信息)

4.0.1 forms 组件能完成的事情

- 渲染 html 页面
- 数据校验
- 展示提示信息

app01 -> forms.py

创建一个form组件

```
from django import forms
class MyForm(forms.Form):
    username = forms.CharField(min_length = 3, max_length = 8)
    password = forms.CharField(min_length = 3, max_length = 8)
    # EmailField要求数据必须满足邮箱格式:xxx@xx.com
    email = forms.EmailField()
```

4.0.1 forms 常用字段类型

`CharField`	**字符串字段**	`<input type="text">`
`EmailField`	邮箱字段	`<input type="email">`
`IntegerField`	整数字段	`<input type="number">`
`DecimalField`	十进制数字段	`<input type="number">`
`BooleanField`	布尔字段	`<input type="checkbox">`
`ChoiceField`	选择字段	`<select>`
`DateField`	日期字段	`<input type="date">`
`DateTimeField`	日期时间字段	`<input type="datetime-local">`
`FileField`	文件上传字段	`<input type="file">`
`ImageField`	图片上传字段	`<input type="file">`

app01 -> test.py

```
from django.test import TestCase

# Create your tests here.
import os
```

```
if __name__ == '__main__':
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'django_forms.settings')
    import django
    django.setup()


    from app01 import forms
    #校验数据
    # 1. 实例化对象,将数据以字典的形式传入
    form_obj = forms.MyForm({'username':'aaa','password':'123','email':'123'})

    #2. 通过对象调用is_vaild()方法查看数据是否合法
    print(form_obj.is_valid()) #必须要所有的数据全部合法才返回True
    #3. 查看没有通过校验的数据以及错误信息
    print(form_obj.errors)
    #4. 查看通过校验的数据
    print(form_obj.cleaned_data)


    #校验数据 多传或者少传数据怎么办?
    form_obj = forms.MyForm({'username': 'aaa', 'password': '123', 'email':
'123@qq.com','hobby':'sing'})
    # 校验数据时,多传的不影响校验,直接忽略
    print(form_obj.is_valid()) # True
    #少传值会报错 因为email是必传的
    form_obj = forms.MyForm({'username': 'aaa', 'password': '123'})
    print(form_obj.is_valid()) #False
    print(form_obj.errors)#This field is required.
```

4.0.1 forms 组件渲染标签:

只能渲染用户输入的标签(input select radio checkbox),不能渲染提交按钮

urls.py

```
 path('index/',views.index)
```

app01 -> views.py

```
def index(request):
    # 先产生一个空对象
    form_obj = forms.MyForm()

    #直接将空对象传递前端页面进行渲染
    #获取用户信息
    if request.method == 'POST':
        #提交的普通键值对数据都封装在request.POST中
        #可以看成是字典的格式.可以直接进行校验
        form_obj = forms.MyForm(request.POST)
        if form_obj.is_valid():
            return HttpResponse('OK')

    return render(request,'index.html',locals())
```

index.html

```
<body>
<!--novalidate 让浏览器不要渲染样式-->
<form action="" method="post" novalidate>
{#    <p>第一种方式:代码书写极少,封装程度很高,不利于扩展</p>#}
{#    {{ form_obj.as_p }} <!-- P标签套label和input的格式 -->#}
{#    {{ form_obj.as_ul }} <!-- ul标签套label和input的格式 -->#}
{#    {{ form_obj.as_table }} <!-- 直接写label和input的格式 -->#}

{#    <p>第二种方式:可扩展性强,但是需要书写的代码太多 这里的lable是你组件声明中字段的
label</p>#}
{#    {{ form_obj.username.label }}:{{ form_obj.username }}#}
{#    {{ form_obj.password.label }}:{{ form_obj.password }}#}
{#    {{ form_obj.email.label }}:{{ form_obj.email }}#}
    <p>第三种方式: 推荐的方式</p>
    {% for form in form_obj %}
        <p>{{ form.label }}:{{ form }}<span style="color: red">{{ form.errors.0
}}</span></p>

    {% endfor %}
    <input type="submit">

</form>

</body>
```

使用事项:

1.创建空对象,在前端渲染页面

2.校验数据是创建对象,两个对象的名称必须一致,在数据不合法的情况下,可以保留上次的数据这样就可以在基于上次结果的基础上进行修改

--

第三种方式: 推荐的方式

用户名: This field is required.

密码: This field is required.

邮箱: This field is required.

提交

现在的问题是怎么改成中文的提示错误

4.0.1 自定义报错信息

forms.py

```
from django import forms
class MyForm(forms.Form):
    username = forms.CharField(min_length = 3, max_length = 8,label='用户名',
                                    error_messages={
                                        'required': '用户名不能为空',
                                        'min_length':'用户名最小3位',
                                        'max_length': '用户名最大8位'
                                    })
    password = forms.CharField(min_length = 3, max_length = 8,label='密码',
                                    error_messages={
                                        'required': '密码不能为空',
                                        'min_length':'密码最小3位',
                                        'max_length': '密码最大8位'
                                    })
    # EmailField要求数据必须满足邮箱格式:xxx@xx.com
    email = forms.EmailField(label='邮箱',
                             error_messages={
                                 'required':'邮箱不能为空',
                                 'invalid':'邮箱格式不正确'
                             })
```

4.0.1 钩子函数

在特定的节点自动触发完成响应操作

类似于forms的第二道关卡,能够让我们自定义校验规则

- **局部钩子**
  - 对单个字段进行校验

```
class MyForm(forms.Form):
    .....

    #局部钩子: 用户名不能包含666
    def clean_username(self):
        # 获取用户名，能到这里进行校验,数据已经通过了第一道关下
        username = self.cleaned_data.get('username')
        if '666' in username:
            #添加报错信息
            self.add_error('username','用户名不能包含666')
        #返回钩子的数据
        return username
```

- **全局钩子**
  - 对多个字段进行校验

```
class MyForm(forms.Form):
     .....

    re_password = forms.CharField(min_length=3, max_length=8, label='确认密码',
                                  error_messages={
                                      'required': '确认密码不能为空',
                                      'min_length': '确认密码最小3位',
                                      'max_length': '确认密码最大8位'
                                  })

    #全局钩子:校验两次密码和确认密码 是否一致
    def clean(self):
        password = self.cleaned_data.get('password')
        re_password = self.cleaned_data.get('re_password')

        if password and re_password and password != re_password:
            self.add_error('re_password', "两次密码不一致")

        return self.cleaned_data
```

测试

第三种方式: 推荐的方式

用户名: admin666 用户名不能包含666

密码: 1234

确认密码: 123 两次密码不一致

邮箱: 123@qq.com

提交

4.0.1 forms 组件的参数说明

Django Forms 组件的字段（Field）有许多参数可以配置，这些参数用于控制字段的行为、验证规则、显示方式等。以下是一些常用的字段参数：

1. **通用参数（适用于大多数字段）**

- `label` (str): 字段的标签（在表单中显示的文本）。如果未提供，Django 会根据字段名自动生成（将下划线替换为空格并首字母大写）。
- `required` (bool): 字段是否必须填写（默认为 True）。如果设置为 False，则字段可以为空。
- `initial` (any): 字段的初始值（当表单未绑定数据时显示）。
- `help_text` (str): 字段的帮助文本（显示在字段下方，用于说明）。
- `error_messages` (dict): 自定义错误消息。可以覆盖默认的错误消息。字典的键是错误类型（如 'required', 'invalid', 'max_length' 等），值是要显示的错误消息字符串。
- `validators` (list): 自定义验证器函数列表。每个验证器是一个可调用对象，接收字段值并验证，如果验证失败则抛出 `ValidationError`。
- `disabled` (bool): 如果为 True，则字段在表单中显示为禁用状态（用户无法编辑）。

- `widget` (Widget): 指定渲染字段的 Widget 类。通过 Widget 可以控制字段在 HTML 中的渲染方式（如输入框、下拉框等）以及 HTML 属性（如 class, placeholder 等）。
- `localize` (bool): 是否本地化字段（用于日期、数字等字段的本地化显示和输入）。
- `label_suffix` (str): 覆盖表单的 `label_suffix`（标签后缀，默认为冒号 ":"）。

2. **特定字段的参数**

字符类型字段（CharField, EmailField, URLField, 等）

- `max_length` (int): 字段的最大长度（字符数）。如果设置了，Django 会验证输入的长度不超过此值。
- `min_length` (int): 字段的最小长度（字符数）。如果设置了，Django 会验证输入的长度不小于此值。
- `strip` (bool): 是否去除输入的前后空白（默认为 True）。对于 `CharField`，通常建议保留为 True。

数值类型字段（IntegerField, DecimalField, FloatField）

- `max_value` (int/float): 允许的最大值。
- `min_value` (int/float): 允许的最小值。

日期时间字段（DateField, DateTimeField, TimeField）

- `input_formats` (list): 可接受的日期/时间格式列表（字符串格式）。例如：`['%Y-%m-%d', '%m/%d/%Y']`。

选择字段（ChoiceField, MultipleChoiceField）

- `choices` (iterable): 字段的选项。通常是一个由二元组（值, 显示文本）组成的列表或元组。例如：`[('M', 'Male'), ('F', 'Female')]`。

文件字段（FileField, ImageField）

- `max_upload_size` (int): 上传文件的最大大小（字节）。注意：实际的文件大小验证通常需要在视图中进行，因为表单验证可能无法完全控制。
- `allow_empty_file` (bool): 是否允许空文件（默认为 False）。

五、作业

5.1 完成本天的案例